1 Key code components

Basic description of the components of the code (probably each will form its own namespace).

I. **mesh**: Will form a namespace which will accommodate classes for the mesh structure. Keep in mind that there will also be a structure that explicitly generates a regular mesh $M \times N$ with a given genome.

II. **basis functions**: Block dedicated to the class that has the edge-detection algorithm, divergence-conforming basis-function class that manages the indexing of the basis functions. Next there will be concrete variants of the divergence-conforming basis functions called rooftop basis functions. Two variants will exist: one that specifically works with the genome of the mesh structure, and a second that ignores the genome and rapidly generates the appropriate structures.

III. **MoM strategy**: A more general name would be "MoM strategy," because it will not only include the impedance-matrix assembly process—there are solvers (iterative, e.g. CGFFT) that do not require the impedance matrix. Thus this namespace will be divided into two parts: `WithImpedanceMatrix` and `WithoutImpedanceMatrix`. The top-level class will decide which strategy to use based on user input and the desired output (impedance matrix, current vector, etc.). The input will be the appropriate excitation.

   i. **memory manager**: Decides the maximum sizes of blocks, vectors, and matrices for subsequent planners.

   ii. **impedance-matrix planner**: Contains several variants of the planning process. Each class produces a structure that accommodates any information needed for the next stage of assembly—basically the indices of the matrix that need to be computed in each iteration and how.

   iii. **impedance-matrix assembler**: Takes the precomputed chunks from the planner and assembles the desired components of the impedance matrix. Consists of a core algorithm specialized in computing the actual impedances.

   iv. **MATLAB's built-in solver**: Produces the final current vector based on the specified excitation.

   v. **Green's-matrix planner**: Similar to the impedance-matrix planner; it determines which indices to generate at once.

   vi. **Green's-matrix assembler**: Assembles the matrix representation of the Green's operator in a format acceptable for subsequent calculations.

   vii. **conjugate-gradient solver**: Solves for the currents given an excitation using the conjugate-gradient method. The preconditioner will be implemented in a separate class.

IV. **antenna operators**: Likely contains the excitation vector (the input to the MoM), represented as the planar-wave operator matrix. This namespace will also include RCS, far-field operators, input impedance, etc.

V. **visualization tools?**

2 Code structure

Here we will expand the whole code structure down to a single function, each will has a brief description what it does. This will not only clarify the code oritentation, it will also motivate each component existence and usage, which should be homogenous with SRP. It will also extinct the duplicity and wrong context of the component. Each (sub)subsection will represent namespace. Then each of the top-level enumerates will represent class names and the subsequent indented enumerated will list all methods of the class.

2.1 `mesh`

All classes asociated with the mesh generation will be placed here. Up on these structures will be placed the classes from the basis function, everything about the geometry of the problem will live here. Therefore we will introduce a structure(s) that will form a bridge to the basis function class.

I. `UniformRectPrimitive`: This is a entry structure that will hold the dimensions of the elementary cell and how many are those in each direction. As well it will hold the genome of the mesh (binary table). The basic unit will be 1 m for now, in future there will be also a posibility to use relative unit system.

   i. `patchCountX`

   ii. `patchCountY`

   iii. `patchCount`: Returns the total number of patches in the mesh.

   iv. `patchCountGenome`: Return the total minus the genome sum.

   v. `patchSizeX`

   vi. `patchSizeY`

   vii. `meshSize`: Returns the geometrical size of the mesh in a pair.

   viii. `patchIndices`: Returns all indices of the patches of the mesh as a 1D vector. Basically used for the mesh generation.

   ix. `patchIndicesGenome`: The same as `patchIndices`, but returns only those where the genome is 1, keeps the indexing.

II. `UniformRect`: Takes the `UniformRectPrimitive` and places the mesh in 3D according to the bottom-left corner (in the future add more variety of rect origins), as well three euler angles which will orient the rectangular plate. The output of this structure is the universal Mesh, but with optimal basis function ordering (important for best acceleration techniques performance).

   i. `generateMesh`: Outputs the mesh of the primitive with optimal ordering of the faces.

III. `Mesh`: The central structure of the `mesh` namespace will hold the mesh structure in two arrays, so by a list of vertices and list of faces (each face will be connected to n vertices). There is a small problem, if the interaction symmetry algorithm has to be optimally perfoment we will prefer some determined scheme for the face ordering because it will influence the basis function ordering.

   i. `vertices`

   ii. `faces`

   iii. `count`: Return 1, here is due to the exchange with `MeshCluster` which will has `count` implemented.

IV. `MeshCluster`: Multiple `Mesh` structures can be accommodate here. The reindexing of the vertices and faces is automatical. Each mesh is represented as a element of a cell. The point is that the `MeshCluster` is interchangible with the `Mesh` class.

   i. `insert`

   ii. `remove`

   iii. `vertices`

   iv. `faces`

   v. `count`: Returns the total number of `Mesh` instances in the cell. Useful for iterating through the cluster.

V. `utils`: Namespace that bundles functions that operates with the `Mesh` and `MeshCluster` objects

   i. `generateInnerEdges`: Function that will return a 2 x N array of patch indices sorted (for better acceleration performance) from given mesh object.

2.2 basis_functions

As described in the section of key components, here will be placed all classes that generates from the `Mesh` or `MeshCluster` the two key structures: indices list and points. The indices list will be generated for divergence conforming basis functions directly from the mesh based on the edge detection algorithm. Each inner edge of the mesh is associated with one divergence conforming basis function.

I. `BasisFunctions`: Base class for all types of basis functions. Purely abstract class.

II. `UniformRectDivergenceComforming`: This is a special case of the divergence comforming basis functions based on uniformly discretized mesh into rectangular patches, other type very popular are RWG basis functions.

   i. `constructor`: Takes the `Mesh` or `MeshCluster` instance and saves a reference.

   ii. `generateIndices`: calls the function `generateInnerEdges` from `mesh/utils`

   iii. `count`: Returns the total number of basis funcitons.

   iv. `countDir`: Returns the number of basis funcitons in given direction.

   v. `faces`: Returns sorted faces first the X direction then Y direction.

   vi. `facesCell`: For each mesh in cluster new cell.

   vii. `facesCellDir`: Returns only specific section of the basis functions based on the direction.

   viii. `meshIDMask`: Generates a mask ID mask, each of the basis functions (column of the structure), is given a number based on its mesh location. Used later for mesh size adjustment.

   ix. «TEMP» `meshCount`: Returns the size of the cluster. I am not sure if it stays here, absolutely not key here.

III. `RooftopBasisFunctions`: This class derives from the `UniformRectDivergenceComforming` and generates the point matrix for each rooftop based on the indices of the basis functions, one by one.

IV. `FastRooftopBasisFunctions`: This one does not derive from the `UniformRectDivergenceComforming`, the input is the `UniformRectPrimitive` it is used for the superfast CG-FFT algorithms.